CS 122A Custom Laboratory Project Report

Multi-Nodal Embedded Security System

Noah Wisner, Josh, Alex

6/10/2026

Introduction

The goal of this project was to develop a smart security monitoring system using multiple Raspberry Pi Pico microcontrollers, CAN bus communication, an FPGA-driven RGB LCD display, using event-driven design. The system was designed to monitor multiple security zones, collect information from various sensors, and provide a centralized interface for viewing system status and responding to security events.

The system consists of sensor nodes connected through a CAN network to a central controller. Sensor nodes monitor environmental and security-related conditions and report status information to the central controller. The controller maintains the overall system state, processes incoming events, records system activity, and updates a dashboard displayed on the RGB LCD. The system supports multiple security states, including disarmed, armed, alert, and fault conditions. An alarm output is activated when security events occur while the system is armed, and heartbeat monitoring is used to detect communication failures or offline nodes.

A major focus of the project was the event-driven architecture. Sensor updates, communication events, and user actions are converted into system events that are processed by a centralized state machine. This approach allows the system to react consistently to changing conditions while maintaining a clear separation of tasks making the system easy to modify for future iterations.

Several features in the initial proposal were not fully implemented in time. Touchscreen was moved to a stretch goal early on due to time constraints. This current system must be driven through the serial monitor for user input. Camera integration was not fully completed due to hardware constraints. The central controller only has two SPI ports that were required by the FPGA and the CAN Controller (Pico does not natively support CAN so the controller converts it into SPI). Additionally the support for multiple distributed nodes was planned but due to hardware failure (PICO failure) testing and integration is incomplete. RTOS was originally

planned and working for our midpoint demos but complications with LVGL being interrupted caused issues, the TA recommended the feature to be dropped to fit our timeline.

Elements of Complexity

Implemented

- CAN Communication
- Heartbeat and Fault Detection
- Central Event-Driven Logic Execution
- Event Logging
- Real-Time Dashboard

Not implemented/Incomplete

- Multi-Zone Support
    - The system logic works, the dashboard would need to be modified. Stopped by hardware failure.
- Touch Screen
    - Time constraint.
- Camera Integration
    - Not enough native SPI support on PICO. Not enough time for a hardware solution.

User Guide

The system is designed to continuously monitor multiple security zone(s) and provide real-time status through a dashboard. Under normal use, sensor nodes collect environmental and security-related information and transmit updates to the central controller. The controller processes incoming events, maintains the current security state, and updates the outputs accordingly.

When the system is powered on, the controller initializes all subsystems and enters the DISARMED state. The dashboard will display the current system state, zone information, connection status, and recent system events.

Sensor nodes will begin transmitting heartbeat messages and sensor updates. Zones that are communicating properly will appear online. If a node fails to communicate within the expected time period, the system will identify that zone as offline and generate a fault event.

The original project design included touchscreen controls for arming, disarming, and acknowledging alarms. Due to integration and time constraints, touchscreen functionality was not completed in time. Instead, the user interactions through the serial monitor connected to the

master controller. Commands entered through the serial monitor are converted into events and processed by the security state machine.

The following actions are available:

- Arm the system
- Disarm the system
- Acknowledge an active alert
- Generate test events for debugging and demonstration purposes

Although physical buttons were considered for the demonstration, serial commands were used because they provide a more reliable and visible demonstration of system behavior during testing and evaluation and illustrate the modularity of the code.

The system operates in four primary states.

- DISARMED
    - The system monitors sensors and records activity but does not activate alarms when security events occur.
- ARMED
    - The system actively monitors all zones. Security events such as motion detection or door activation may cause the system to enter the ALERT state.
- ALERT
    - An alarm condition has been detected while the system is armed. The system activates the alarm output and records the event in the event log. The alarm can only be stopped by an ACKNOWLEDGE event or a FAULT.
- FAULT
    - A communication failure was detected. Fault conditions are displayed on the dashboard and recorded in the event log. All states can jump to FAULT.

The dashboard displays:

- Current system state
- Zone status information
- Sensor activity indicators
- Node online/offline status
- Recent event log entries
- Active alarms or fault conditions

All significant events are also recorded in the event log. Recent events are displayed directly on the dashboard allowing users to observe system activity and react.

The system includes an audible alarm output. When an alert condition is detected while the system is armed, the alarm is activated. The alarm remains active until the condition is acknowledged or cleared according to the system state machine.

During normal operation, the user arms the system through the serial monitor and observes system status through the dashboard. Sensor nodes continuously report information to the controller. If a monitored condition is detected, the system generates an alert, activates the alarm output, and records the event. Communication failures are automatically detected through heartbeat monitoring and are reported as fault conditions. The system is intended to operate with minimal user interaction, acting as a centralized monitoring and alerting platform for distributed security zones and current setup works well with patrolling security.

Hardware Components Used

- 40-pin TFT Display - 480x272 with Touchscreen
- 2x Raspberry Pico 2

Software Libraries Used

- Allen Knight's Icesugar-Pro-Framebuffer
- LVGL

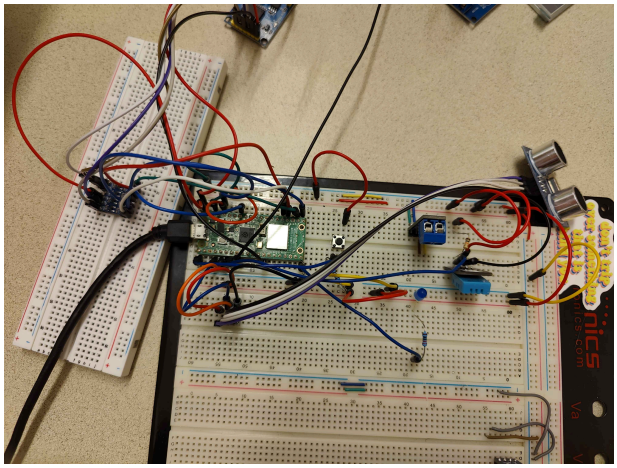
Protocols Used

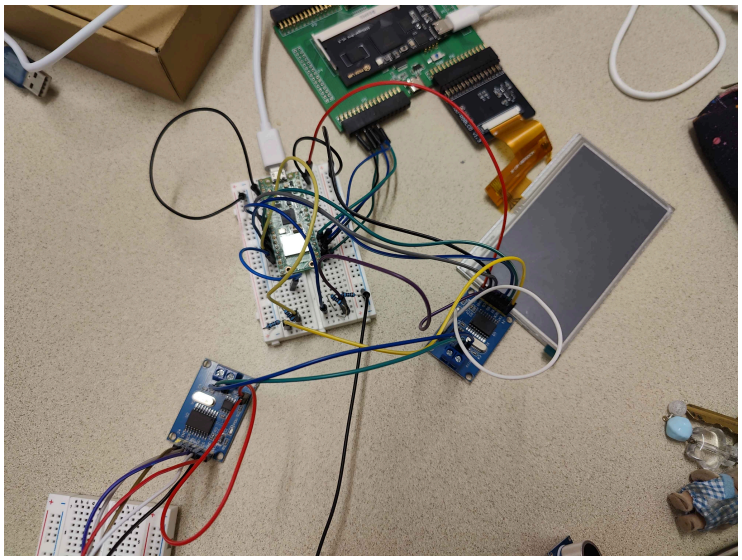
- CAN
    - CAN is used in between the zone nodes.
- SPI
    - SPI is used for PICO->FPGA communication and PICO->CAN conversion

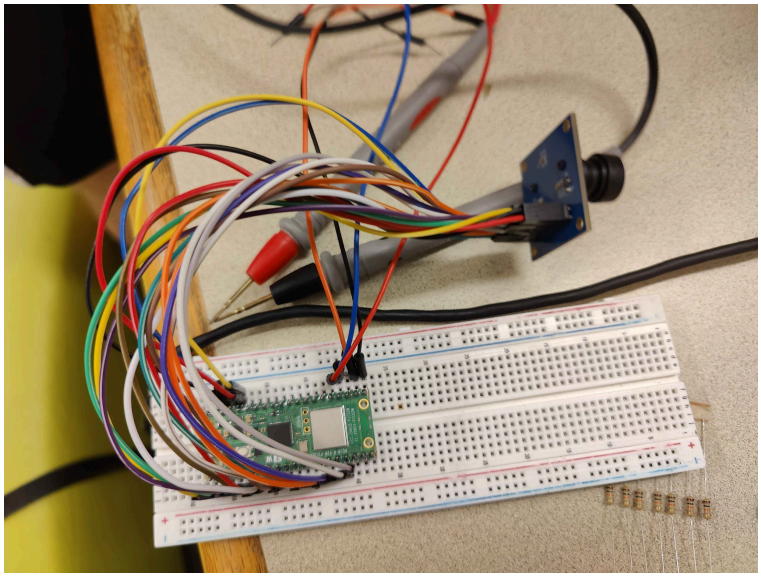
Requirements

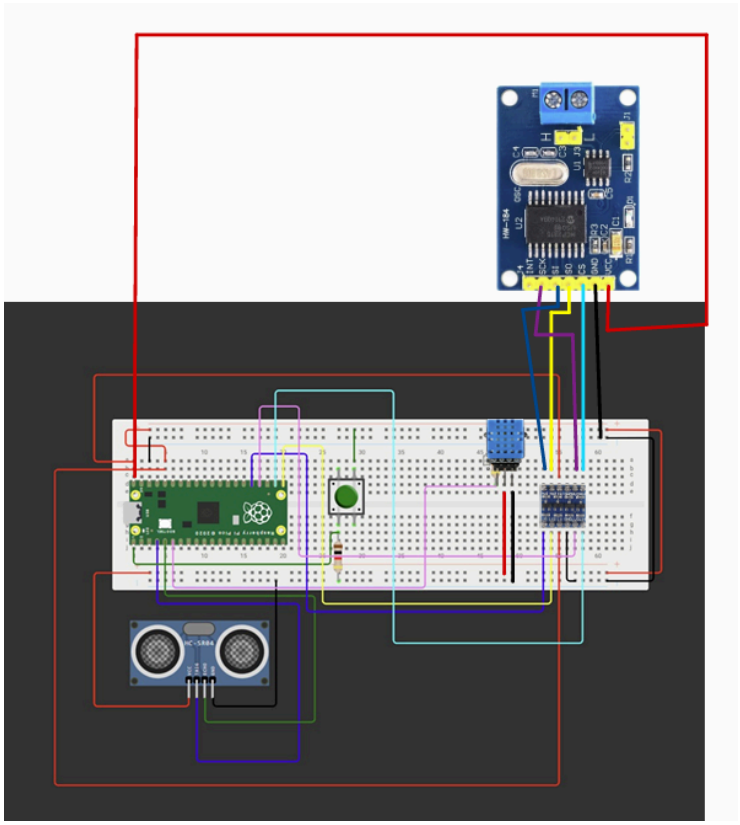
Wiring Diagram

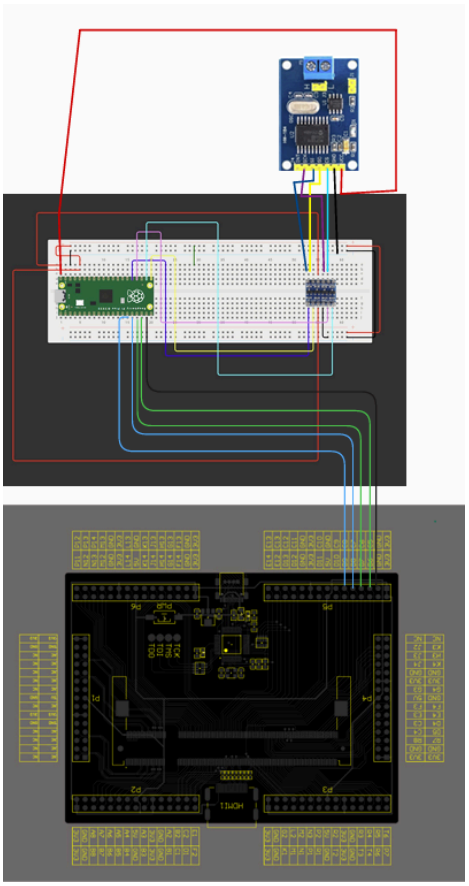
- Draw a wiring diagram to show the physical setup of the system.
- Take a digital photograph of your physical setup and place it next to or below the diagram.





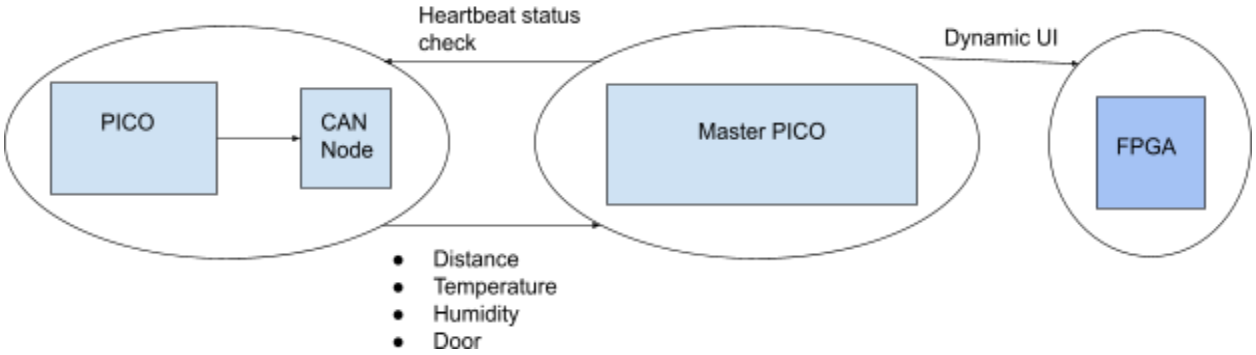






Design Diagram

Simplified System Overview



Transition Tables for System States

DISARMED

Event	Action	Next State
ARM	Log "System Armed"	ARMED
DISARM	Ignore	DISARMED
ZONE_TRIGGERED	Log only	DISARMED
ZONE_CLEARED	Ignore	DISARMED
ALERT_ACK	Ignore	DISARMED
HEARTBEAT_TIMEOUT	Log fault	FAULT
ZONE_RECOVERED	Log recovery	DISARMED

ARMED

Event	Action	Next State
ARM	Ignore	ARMED
DISARM	Disarm	DISARMED
ZONE_TRIGGERED	Alarm ON	ALERT

ZONE_CLEARED	Ignore	ARMED
ALERT_ACK	Ignore	ARMED
HEARTBEAT_TIMEOUT	Log fault	FAULT
ZONE_RECOVERED	Ignore	ARMED

ALERT

Event	Action	Next State
ARM	Ignore	ALERT
DISARM	Alarm OFF	DISARMED
ZONE_TRIGGERED	Log additional trigger	ALERT
ZONE_CLEARED	Log clear	ALERT
ALERT_ACK	Alarm OFF	ARMED
HEARTBEAT_TIMEOUT	Log fault	FAULT
ZONE_RECOVERED	Ignore	ALERT

FAULT

Event	Action	Next State
ARM	Ignore	FAULT
DISARM	Ignore	FAULT
ZONE_TRIGGERED	Ignore	FAULT
ZONE_CLEARED	Ignore	FAULT
ALERT_ACK	Ignore	FAULT
HEARTBEAT_TIMEOUT	Ignore	FAULT

ZONE_RECOVERED	Clear fault	DISARMED

AI Usage

- Generated position values for LVGL components to save time. All objects, general layout and functions were not generated but position values and align values were assisted as multizone was dropped. Multiple times through debugging it was used to help fix edge conditions. - Noah Wisner.
- Generated MCP initialization drivers and CAN Bus Configuration, as well as debugging code that used the serial monitor to diagnose CAN status. - Alexander Montano-Leon
- Generated tests via debugging with the serial monitor for diagnosing task and sensor functionality. Added comments to help partners navigate code design. - Joshua Kibreab

Acknowledgements